

30. dashboard + HTTP API

solar_ws0129-main

2026-07-29

Contents

1 対象	4
2 このファイルを読む理由	4
3 呼び出し関係	4
3.1 どこから呼ばれるか	4
3.2 次に読むと理解が進むファイル	4
3.3 同一リポジトリ内の主要依存	4
4 ソース冒頭の把握	4
4.1 代表コード断片	4
5 主要構造の読み取り	5
5.1 主要クラス・関数	5
5.2 ファイルを上から読んだときの定義順	6
5.3 import 群の詳細	6
6 このファイルを読む前に必要な基礎知識	7
6.1 プログラム、プロセス、メモリ、オブジェクトを区別する	7
6.2 名前、代入、型、参照	7
6.3 関数、引数、戻り値、スコープ	8
6.4 module、package、import、console_scripts	8
6.5 例外、try/finally、with、資源解放	8
6.6 class、独自クラス、インスタンス、self、継承	9
6.7 先頭アンダースコア、__init__、名前付けの作法	9
6.8 list、dict、deque、NumPy 配列、pandas DataFrame	9
6.9 rclpy、rcl、rmw、DDS/RTSPS の層	10
6.10 ROS graph、Node、topic、service、action、parameter	10
6.11 callback、timer、Executor、spin、Callback Group	10
6.12 rqt_graph、ros2 CLI、roslaunch をいつ使うか	11
6.13 天候、route、補間、時刻、単位	11
6.14 制御、状態、入力、モデル予測制御 MPC	11

7 関数・クラスを上から順に解説	12
7.1 L19 クラス DashboardHandler	12
7.2 L20 関数 DashboardHandler.__init__	13
7.3 L24 関数 DashboardHandler.do_GET	14
7.4 L33 関数 DashboardHandler.log_message	15
7.5 L37 関数 DashboardHandler._send_json	15
7.6 L45 関数 DashboardHandler._send_metrics	16
7.7 L55 クラス DashboardNode	17
7.8 L56 関数 DashboardNode.__init__	19
7.9 L156 関数 DashboardNode.destroy_node	22
7.10 L165 関数 DashboardNode.get_state	23
7.11 L169 関数 DashboardNode.get_prometheus_metrics	23
7.12 L236 関数 DashboardNode._update	25
7.13 L241 関数 DashboardNode._on_speed_cmd	26
7.14 L244 関数 DashboardNode._on_upper_speed_cmd	27
7.15 L247 関数 DashboardNode._on_speed_meas	27
7.16 L250 関数 DashboardNode._on_throttle_cmd	28
7.17 L253 関数 DashboardNode._on_drive_mode	29
7.18 L256 関数 DashboardNode._on_soc	29
7.19 L264 関数 DashboardNode._set_estimated_soc	30
7.20 L273 関数 DashboardNode._on_tb	31
7.21 L276 関数 DashboardNode._on_ibatt	32
7.22 L279 関数 DashboardNode._on_vbatt	32
7.23 L282 関数 DashboardNode._on_s_km	33
7.24 L285 関数 DashboardNode._on_status	33
7.25 L301 関数 DashboardNode._on_env	35
7.26 L313 関数 DashboardNode._on_metrics	36
7.27 L333 関数 DashboardNode._on_upper_plan	37
7.28 L344 関数 DashboardNode._on_lower_plan	38
7.29 L355 関数 DashboardNode._on_sys_state	39
7.30 L358 関数 DashboardNode._on_sys_diag	40
7.31 L361 関数 DashboardNode._on_mpc_state	40
7.32 L364 関数 DashboardNode._on_health	41
7.33 L367 関数 DashboardNode._on_gps	42
7.34 L373 関数 DashboardNode._on_path	42
7.35 L379 関数 DashboardNode._init_dummy	43
7.36 L395 関数 DashboardNode._tick_dummy	44
7.37 L409 関数 main	46
7.38 設定値と入力パラメータ	46
7.39 ROS topic I/O	47
8 処理の流れ	47

1 対象

項目	内容
ファイル	mpc_solarcar/dashboard_node.py
ソース SHA-256	26286dcb019288a13c2e17be4574c311b8bc2bb5e0268544165d6519a5cba443
種別	Python
区分	runtime node
役割	planner と vehicle の現在値を ROS から受け、HTTP サーバと dashboard frontend へ渡す。
起動文脈	可視化の中心ノード。

2 このファイルを読む理由

planner と vehicle の現在値を ROS から受け、HTTP サーバと dashboard frontend へ渡す。

- /api/state と /metrics を持つ。
- ROS topic を直接 browser へ出すのではなく、Node 内 state に集約する。

3 呼び出し関係

3.1 どこから呼ばれるか

- launch/solarcar_sim.launch.py
- mpc_solarcar/live_launch.py
- launch/solar_measurement.launch.py

3.2 次に読むと理解が進むファイル

- 特記事項なし。

3.3 同一リポジトリ内の主要依存

- 同一リポジトリ内の明示 import は少ない。

4 ソース冒頭の把握

モジュール docstring または先頭意図の要約:

モジュール docstring は明示されていない。

4.1 代表コード断片

```
from ament_index_python.packages import get_package_share_directory

class DashboardHandler(SimpleHTTPRequestHandler):
    def __init__(self, *args, node=None, directory=None, **kwargs):
        self._node = node
```

```

    super().__init__(*args, directory=directory, **kwargs)

def do_GET(self):
    if self.path.startswith('/api/state'):
        self._send_json(self._node.get_state())
        return
    if self.path.startswith('/metrics'):
        self._send_metrics(self._node.get_prometheus_metrics())
        return
    super().do_GET()

def log_message(self, format, *args):
    # quiet
    return

def _send_json(self, data):

```

5 主要構造の読み取り

主要クラスは DashboardHandler, DashboardNode。主要関数は do_GET, log_message, destroy_node, get_state, get_prometheus_metrics, main。ROS パラメータ宣言は 5 件。ROS I/O は publisher 0 件、subscription 21 件。

5.1 主要クラス・関数

行	種別	名前	補足
19	class	DashboardHandler	bases: SimpleHTTPRequestHandler
55	class	DashboardNode	bases: Node
20	function	__init__	args: self
24	function	do_GET	args: self
33	function	log_message	args: self, format
37	function	_send_json	args: self, data
45	function	_send_metrics	args: self, payload
56	function	__init__	args: self
156	function	destroy_node	args: self
165	function	get_state	args: self
169	function	get_prometheus_metrics	args: self
236	function	_update	args: self, key, value
241	function	_on_speed_cmd	args: self, msg
244	function	_on_upper_speed_cmd	args: self, msg
247	function	_on_speed_meas	args: self, msg
250	function	_on_throttle_cmd	args: self, msg
253	function	_on_drive_mode	args: self, msg
256	function	_on_soc	args: self, msg
264	function	_set_estimated_soc	args: self, value
273	function	_on_tb	args: self, msg
276	function	_on_ibatt	args: self, msg
279	function	_on_vbatt	args: self, msg
282	function	_on_s_km	args: self, msg
285	function	_on_status	args: self, msg

301	function	_on_env	args: self, msg
313	function	_on_metrics	args: self, msg
333	function	_on_upper_plan	args: self, msg
344	function	_on_lower_plan	args: self, msg
355	function	_on_sys_state	args: self, msg
358	function	_on_sys_diag	args: self, msg
361	function	_on_mpc_state	args: self, msg
364	function	_on_health	args: self, msg
367	function	_on_gps	args: self, msg
373	function	_on_path	args: self, msg
379	function	_init_dummy	args: self, path, rate_hz
395	function	_tick_dummy	args: self
409	function	main	

5.2 ファイルを上から読んだときの定義順

行	内容
19	クラス DashboardHandler を定義する。
55	クラス DashboardNode を定義する。
409	関数 main を定義する。
417	条件 <code>__name__ == '__main__'</code> を判定し、真なら内部処理を行う。

5.3 import 群の詳細

行	import 文	このファイルで読む理由
1	<code>import json</code>	manifest、checkpoint、UDP payload をやり取りするため。このファイル内での主な使用位置は L38。
2	<code>import math</code>	Python 標準のスカラー数値関数と定数を使うため。実際に参照する名前と行は使用位置欄で確認する。このファイル内での主な使用位置は L212。
3	<code>import os</code>	パス、環境変数、プロセス外部状態を扱うため。このファイル内での主な使用位置は L68, L70, L71。
4	<code>import threading</code>	threading モジュールを利用するため。このファイル内での主な使用位置は L73, L150。
5	<code>import time</code>	wall/monotonic time に基づく周期制御や freshness 判定を行うため。このファイル内での主な使用位置は L75, L215, L239, L261, L262, L268, L299, L311, ...。
6	<code>from functools import partial</code>	functools から partial を読み込み、このファイルの処理を組み立てるため。このファイル内での主な使用位置は L147。
7	<code>from http.server import SimpleHTTPRequestHandler, HTTPServer</code> <code>import SimpleHTTPRequestHandler, HTTPServer</code> <code>ThreadingHTTPServer</code>	http.server から SimpleHTTPRequestHandler, Thread- ingHTTPServer を読み込み、このファイルの処理を 組み立てるため。このファイル内での主な使用位置 は L19, L149。
9	<code>import rclpy</code>	ROS 2 Python ノードとして起動・spin するため。こ のファイル内での主な使用位置は L410, L412, L414。

10	<code>fromrclpy.nodeimportNode</code>	ROS 2 ノード本体の基底クラスとして使うため。このファイル内での主な使用位置は L55。
12	<code>fromstd_msgs. msgimportFloat32, Float32MultiArray,String</code>	Float32、Bool、String などの標準 ROS message で値を publish または subscribe するため。このファイル内での主な使用位置は L117, L118, L119, L120, L121, L122, L123, L124, ...。
13	<code>fromsensor_msgs. msgimportNavSatFix</code>	GPS 緯度経度高度を ROS topic でやり取りするため。このファイル内での主な使用位置は L136, L367。
14	<code>fromnav_msgs.msgimportPath</code>	将来軌跡を dashboard や logger へ出すため。このファイル内での主な使用位置は L137, L373。
16	<code>fromament_index_python. packagesimportget_package_ share_directory</code>	ament_index_python.packages から get_package_share_directory を読み込み、このファイルの処理を組み立てるため。このファイル内での主な使用位置は L67。
381	<code>importpandasaspd</code>	CSV や時刻列の読込・整形・再サンプリングに使うため。このファイル内での主な使用位置は L382。

6 このファイルを読む前に必要な基礎知識

次の章は、構文や ROS 用語を既知と仮定しないための説明である。

6.1 プログラム、プロセス、メモリ、オブジェクトを区別する

ソースファイルはディスク上の文字列であり、それ自体は走っていない。Python 実行可能プログラムがソースを読み、OS がその実行を一つのプロセスとして管理する。プロセスには仮想メモリ、開いているファイル、スレッド、終了コードなどが対応する。

ソースファイル -> Python インタプリタが読み込む -> OS 上のプロセス -> Python オブジェクトをメモリに生成 -> 関数や callback を実行

「メモリ上に生成する」とは、実行中プロセスが使う記憶領域に、その値の型、属性、参照関係を表す実体を用意することである。変数は実体そのものというより、そのオブジェクトを指す名前として理解すると Python の代入が読みやすい。

```
node = MPCNode()
alias = node
# node と alias は同じオブジェクトを参照する。
```

一つのプロセスには複数スレッドを持たせられる。スレッドは同じプロセスのメモリを共有するため、受信 callback と最適化 callback が同じ self.z を書き換える場合は実行順と排他を検討する必要がある。

根拠資料

- [Python 公式チュートリアル: Classes](#)

6.2 名前、代入、型、参照

Python の代入文は、右辺を先に評価し、その結果のオブジェクトへ左辺の名前を結び付ける。‘x = f()’では、まず f を呼び、その戻り値が得られてから x が更新される。

‘float(...)’、‘int(...)’、‘bool(...)’は型名を呼び出して値を変換する。外部 CSV、YAML、ROS parameter から来た値は型が期待どおりとは限らないため、このリポジトリでは明示変換が多い。

‘None’は値が無いことを示す単一のオブジェクト、‘math.nan’は浮動小数点値ではあるが有効な数値ではないことを示す。両者は用途が異なるため、‘is None’と ‘math.isfinite’を使い分ける。

根拠資料

- [Python 公式チュートリアル: Classes](#)

6.3 関数、引数、戻り値、スコープ

‘def’は関数本体をその場で実行する文ではない。関数オブジェクトを作り、その名前を現在の名前空間へ登録する。‘f’は関数そのもの、‘f()’はその関数を呼んだ結果である。

仮引数は関数定義側の受け取り名、実引数は呼出側から渡す値である。位置引数、キーワード引数、既定値付き引数、‘*args’、‘**kwargs’は渡し方が異なる。

```
def clip_speed(value, minimum=0.0, maximum=110.0):  
    return min(maximum, max(minimum, value))  
  
v = clip_speed(120.0, maximum=90.0)
```

関数内で作った通常の名前はローカルスコープに属する。メソッドが‘self.z’を更新すればオブジェクトの状態が残るが、単に‘z’を更新しただけならその呼出しのローカル値である。

根拠資料

- [Python 公式チュートリアル: Classes](#)

6.4 module、package、import、console_scripts

Python ファイルは module として読み込める。複数 module をディレクトリにまとめたものが package である。‘from .model import SolarCarModel’の先頭の点は、現在と同じ package 内の model module を指す相対 import である。

import 時にはファイルのトップレベル文が上から一度実行される。‘def’や‘class’は関数・クラスを登録するが、その本体の通常処理は呼び出すまで走らない。

setuptools の‘console_scripts’は、端末で使う実行可能名と‘package.module:function’を対応付けるインストール時メタデータである。生成される実行用ラッパーは module を import し、指定関数を呼び、戻り値を終了コードとして扱う小さな入口である。

```
ROS launchのexecutable名 -> install済みconsole script -> mpc_solarcar.mpc_nodeをimport ->  
main()を呼ぶ
```

根拠資料

- [setuptools 公式: Entry Points / Console Scripts](#)
- [Python 公式チュートリアル: Classes](#)

6.5 例外、try/finally、with、資源解放

例外は通常の戻り値とは別経路で異常を呼出元へ伝える。‘try/except’は想定した異常を処理し、‘finally’は成功・失敗にかかわらず後始末を行う。

‘with open(...) as f:’は context manager を使い、ブロックを出るとファイルを閉じる。CSV やログの破損を避けるため、開いた資源の所有者と閉じる場所を明確にする。

‘except Exception: pass’はノードを止めない利点がある一方、入力異常を隠して原因追跡を難しくする。安全に関係する値では、少なくとも頻度制限付き警告、異常カウンタ、fallback 状態の publish を検討する。

6.6 class、独自クラス、インスタンス、self、継承

クラスは、保持するデータと、そのデータを扱う関数を一つの型としてまとめる設計単位である。「独自クラス」とは Python や ROS 2 が最初から用意した型ではなく、このプロジェクトが目的に合わせて定義した新しい型を指す。

‘class MPCNode(Node)’は、rclpy の Node を基底クラスとして継承し、Node が持つ publisher、subscription、timer、parameter などの機能に MPC 固有機能を追加する。丸括弧内は関数引数ではなく基底クラス指定である。

‘MPCNode()’はクラスオブジェクトを呼び出して新しいインスタンスを作る。Python は新しい実体を用意し、その実体を第 1 引数 self として ‘__init__’へ渡す。‘self’は予約語ではなく慣習的な名前だが、この慣習を守ることでコードの意味が共有される。

```
class Counter:
    def __init__(self):
        self.value = 0

    def increment(self):
        self.value += 1

counter = Counter()
counter.increment()
```

‘super().__init__(...)’は継承元の初期化処理を呼ぶ。MPCNode ではこれを省くと ROS ノードとして必要な内部実体が作られず、‘create_publisher’などを正しく使えない。

根拠資料

- [Python 公式チュートリアル: Classes](#)
- [ROS 2 Humble 公式: Understanding nodes](#)

6.7 先頭アンダースコア、__init__、名前付けの作法

‘self_step_solar’の先頭 1 個のアンダースコアは「クラス内部で使う実装詳細」という慣習であり、アクセスを強制禁止する機構ではない。外部コードから呼べるが、公開 API として依存しない意思表示である。

‘__init__’の前後 2 個のアンダースコアは Python が定めた特殊メソッド名である。クラス生成時に自動で呼ばれる。任意の名前に前後 2 個を付けて独自仕様を作ることは避ける。

‘_’で始まるローカル変数は、未使用または外部へ見せない意図を示す場合がある。例えば ‘_base_csv’は戻り値として受け取る必要はあるが、その後の処理では使わないことを読者へ示す。

根拠資料

- [Python 公式チュートリアル: Classes](#)

6.8 list、dict、deque、NumPy 配列、pandas DataFrame

list は順序付き可変列、tuple は順序付きで通常変更しない列、dict はキーから値を引く対応表、deque は両端追加・削除が効率的なキューである。どの構造を選ぶかはアクセス方法と更新方法で決まる。

NumPy 配列は同種数値を連続的に扱い、要素ごとの演算、clip、補間、線形代数を簡潔に書く。shape は各次元の要素数であり、速度系列なら通常 ‘(N,)’、候補集団なら ‘(population, N)’となる。

pandas DataFrame は列名を持つ表である。CSV 読込後は列型、欠損、単位、timezone、並び順、重複を明示的に処理しなければ、数値計算が動いても意味が誤る。

6.9 rclpy、rcl、rmw、DDS/RTPS の層

rclpy は Python 利用者向け ROS 2 client library である。利用者の Node、publisher、subscription などを Python API として提供し、その下で共通 C 層の rcl を利用する。

本プロジェクトのPythonコード -> rclpy -> rcl -> rmw -> DDS/RTPS実装 -> ネットワークまたは同一PC 内通信

rcl は言語に依存しない共通 ROS 機能を提供する C API、rmw は ROS 2 と具体的 middleware 実装の境界である。DDS/RTPS 側が探索、serialize、publish/subscribe、request/replyなどを担う。executor の実行モデルは rcl だけで完結せず client library 側にも実装される。

根拠資料

- [ROS 2 Humble 公式: Internal ROS 2 interfaces](#)

6.10 ROS graph、Node、topic、service、action、parameter

ROS graph は、実行中 Node と、それらが持つ publisher、subscription、service、action などの接続関係である。Python クラス、プロセス、ROS graph 上の Node 名は関連するが同一物ではない。

topic は継続データ向けの非同期一方向 publish/subscribe、service は短い request/response、action は時間のかかる目標へ feedback、cancel、result を持たせる。parameter は Node ごとの設定値である。

publisher が送った時点と subscription callback が実行される時点は同じとは限らない。通信遅延、QoS queue、executor 待ちを挟むため、センサ値には timestamp と freshness 判定が必要になる。

根拠資料

- [ROS 2 Humble 公式: Understanding nodes](#)
- [ROS 2 Humble 公式: Topics, Services, Actions](#)
- [ROS 2 Humble 公式: Parameters](#)

6.11 callback、timer、Executor、spin、Callback Group

callback は、メッセージ受信、timer 満了、service request などのイベントが成立した後で Executor から呼ばれる関数である。登録時に 'self._on_speed' と括弧なしで渡すのは、今実行せず後で呼ぶ関数オブジェクトを渡すためである。

Executor は実行可能になった callback を見つけ、Callback Group の条件を確認して実行する。'spin()' は終了要求まで待機・dispatch を続けるため、通常運転中は main の次の行へ戻らない。

MutuallyExclusiveCallbackGroup は同じ group 内の callback を同時実行させない。別 group 間は Multi-ThreadedExecutor で並行実行し得る。group を分けただけでは共有属性への完全な排他にはならないため、複数 group が同じ状態を読む・書く場合は設計確認が必要である。

DDS受信またはtimer満了 -> wait setでready -> Executorが選択 -> Callback Groupが許可 -> worker threadがcallback実行 -> 終了後Executorへ戻る

根拠資料

- [ROS 2 公式: Executors](#)
- [ROS 2 Humble 公式: Internal ROS 2 interfaces](#)

6.12 rqt_graph、ros2 CLI、rosvbag2 をいつ使うか

rqt_graph は接続関係を見る道具であり、数値の正しさや更新周期までは保証しない。起動直後、topic 名変更後、publisher が複数存在する疑いがある時に使う。

```
ros2 node list
ros2 node info /mpc_node
ros2 topic list -t
ros2 topic info -v /planner/speed_cmd
ros2 topic hz /planner/speed_cmd
ros2 topic echo /planner/status
rqt_graph
```

rosvbag2 は topic メッセージを時系列のまま記録・再生する。通信不具合、freshness、再計画 trigger、実車と SILS の差を再現可能にするため、本番前試験では制御入力だけでなく原因となる全 telemetry、status、parameter 情報を記録する。

```
ros2 bag record -o outputs/bags/preflight \
  /vehicle/s_km /vehicle/speed_kmh /vehicle/batt_soc \
  /vehicle/batt_temp_c /vehicle/batt_current_a /vehicle/batt_voltage_v \
  /planner/upper_speed_cmd /planner/speed_cmd /planner/status

ros2 bag info outputs/bags/preflight
ros2 bag play outputs/bags/preflight --clock
```

bag 再生時は QoS 互換性、simulation time、外部 publisher との二重入力に注意する。実車 Node を同時に動かす場合は namespace または remapping で入力源を明確に分離する。

根拠資料

- [ROS 2 Humble 公式: rqt_graph](#)
- [ROS 2 Humble 公式: Recording and playing back data](#)
- [ROS 2 Humble 公式: Understanding nodes](#)

6.13 天候、route、補間、時刻、単位

予報は時刻だけの系列か、時刻と route 距離の 2 次元 grid になり得る。現在時刻 t と距離 s が grid 点の間なら、周囲の値から補間して日射、温度、風を求める。

UTC、現地 timezone、naive datetime を混ぜると数時間のずれが発生する。入力で timezone を確定し、内部比較は UTC、表示は現地時刻という役割分担が安全である。

route 距離の km、速度の km/h と m/s、時間の秒、energy の Wh と J を跨ぐため、変換係数 3.6、3600、1000 の意味を各式で確認する。

6.14 制御、状態、入力、モデル予測制御 MPC

制御対象の内部を表す状態を x 、操作入力を u 、外乱・予報を w とすると、離散モデルは ' $x[k+1] = f(x[k], u[k], w[k])$ ' と書ける。ソーラーカーでは SoC、電池温度、距離、速度などが状態候補、速度目標や駆動トルクが入力候補になる。

$$\min_{u_0, \dots, u_{N-1}} \sum_{k=0}^{N-1} \ell(x_k, u_k, w_k) + V_f(x_N)$$

MPC は現在状態から N ステップ先まで予測し、目的関数と制約を満たす入力系列を求める。ただし実際に適用するのは通常先頭入力だけで、次回は新しい実測状態から再び解く。これが receding horizon である。

予測モデル、目的関数、制約、ホライズン、solver、初期値のどれかが変わると答えも変わる。「MPCを使う」だけでは仕様は決まらず、これらを単位付きで追う必要がある。

根拠資料

- SciPy 公式: [scipy.optimize.minimize](#)

7 関数・クラスを上から順に解説

7.1 L19 クラス DashboardHandler

- 行範囲: L19–L52
- 所属: module 直下
- 定義: `DashboardHandler(bases=SimpleHTTPRequestHandler)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: 特に明示されていない。
- 戻り値の要点: 戻り値が無いか、`return` 文が明示されていない。
- 主なローカル代入名: 特になし。
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、`try` 0

この定義を読むための Python 構文

- `class` 文は新しい型を定義する。丸括弧内は継承する基底クラスである。

上から順の処理

1. 関数 `__init__` を定義する。
2. 関数 `do_GET` を定義する。
3. 関数 `log_message` を定義する。
4. 関数 `_send_json` を定義する。
5. 関数 `_send_metrics` を定義する。

代表コード断片

```
class DashboardHandler(SimpleHTTPRequestHandler):
    def __init__(self, *args, node=None, directory=None, **kwargs):
        self._node = node
        super().__init__(*args, directory=directory, **kwargs)

    def do_GET(self):
        if self.path.startswith('/api/state'):
            self._send_json(self._node.get_state())
            return
```

```

    if self.path.startswith('/metrics'):
        self._send_metrics(self._node.get_prometheus_metrics())
        return
    super().do_GET()

def log_message(self, format, *args):
    # quiet
    return

def _send_json(self, data):
    payload = json.dumps(data).encode('utf-8')
    self.send_response(200)
    self.send_header('Content-Type', 'application/json')
    self.send_header('Content-Length', str(len(payload)))
    self.end_headers()
    self.wfile.write(payload)

def _send_metrics(self, payload):
    body = payload.encode('utf-8')
    self.send_response(200)
    self.send_header('Content-Type', 'text/plain; version=0.0.4; charset=utf-8')
    self.send_header('Cache-Control', 'no-store')
    self.send_header('Content-Length', str(len(body)))
    self.end_headers()
    self.wfile.write(body)

```

7.2 L20 関数 DashboardHandler.__init__

- 行範囲: L20–L22
- 所属: DashboardHandler
- 定義: `__init__(self, *args, node=None, directory=None, **kwargs)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `__init__`, `super`
- 戻り値の要点: 戻り値が無いか、`return` 文が明示されていない。
- 主なローカル代入名: 特になし。
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: `self._node`
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、try 0

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。

上から順の処理

1. `self._node` に `node` の結果を代入する。
2. `super().__init__(...)` を実行する。

代表コード断片

```
def __init__(self, *args, node=None, directory=None, **kwargs):
    self._node = node
    super().__init__(*args, directory=directory, **kwargs)
```

7.3 L24 関数 DashboardHandler.do_GET

- 行範囲: L24–L31
- 所属: DashboardHandler
- 定義: do_GET(self)
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: _send_json, _send_metrics, do_GET, get_prometheus_metrics, get_state, startswith, super
- 戻り値の要点: 戻り値が無いが、return 文が明示されていない。
- 主なローカル代入名: 特になし。
- 読み取る主なインスタンス属性: self._node, self._send_json, self._send_metrics, self.path
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 raise はない。
- 制御構造の規模: 条件分岐 2、ループ 0、try 0

この定義を読むための Python 構文

- 第 1 引数 self は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。

上から順の処理

1. 条件 self.path.startswith('/api/state') を判定し、真なら内部処理を行う。
2. self._send_json(...) を実行する。
3. を返す。
4. 条件 self.path.startswith('/metrics') を判定し、真なら内部処理を行う。
5. self._send_metrics(...) を実行する。
6. を返す。
7. super().do_GET(...) を実行する。

代表コード断片

```
def do_GET(self):
    if self.path.startswith('/api/state'):
        self._send_json(self._node.get_state())
        return
    if self.path.startswith('/metrics'):
        self._send_metrics(self._node.get_prometheus_metrics())
        return
    super().do_GET()
```

7.4 L33 関数 `DashboardHandler.log_message`

- 行範囲: L33–L35
- 所属: `DashboardHandler`
- 定義: `log_message(self, format, *args)`
- `docstring`: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: 特に明示されていない。
- 戻り値の要点: 戻り値が無いか、`return` 文が明示されていない。
- 主なローカル代入名: 特になし。
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、`try` 0

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。

上から順の処理

1. を返す。

代表コード断片

```
def log_message(self, format, *args):  
    # quiet  
    return
```

7.5 L37 関数 `DashboardHandler._send_json`

- 行範囲: L37–L43
- 所属: `DashboardHandler`
- 定義: `_send_json(self, data)`
- `docstring`: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `dumps`, `encode`, `end_headers`, `len`, `send_header`, `send_response`, `str`, `write`
- 戻り値の要点: 戻り値が無いか、`return` 文が明示されていない。
- 主なローカル代入名: `payload`
- 読み取る主なインスタンス属性: `self.end_headers`, `self.send_header`, `self.send_response`, `self.wfile`
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、`try` 0

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。

上から順の処理

1. `payload` に `json.dumps(data).encode('utf-8')` の結果を代入する。
2. `self.send_response(...)` を実行する。
3. `self.send_header(...)` を実行する。
4. `self.send_header(...)` を実行する。
5. `self.end_headers(...)` を実行する。
6. `self.wfile.write(...)` を実行する。

代表コード断片

```
def _send_json(self, data):
    payload = json.dumps(data).encode('utf-8')
    self.send_response(200)
    self.send_header('Content-Type', 'application/json')
    self.send_header('Content-Length', str(len(payload)))
    self.end_headers()
    self.wfile.write(payload)
```

7.6 L45 関数 `DashboardHandler._send_metrics`

- 行範囲: L45–L52
- 所属: `DashboardHandler`
- 定義: `_send_metrics(self, payload)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `encode`, `end_headers`, `len`, `send_header`, `send_response`, `str`, `write`
- 戻り値の要点: 戻り値が無いか、`return` 文が明示されていない。
- 主なローカル代入名: `body`
- 読み取る主なインスタンス属性: `self.end_headers`, `self.send_header`, `self.send_response`, `self.wfile`
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、`try` 0

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。

上から順の処理

1. `body` に `payload.encode('utf-8')` の結果を代入する。
2. `self.send_response(...)` を実行する。
3. `self.send_header(...)` を実行する。
4. `self.send_header(...)` を実行する。
5. `self.send_header(...)` を実行する。
6. `self.end_headers(...)` を実行する。
7. `self.wfile.write(...)` を実行する。

代表コード断片

```
def _send_metrics(self, payload):
    body = payload.encode('utf-8')
    self.send_response(200)
    self.send_header('Content-Type', 'text/plain; version=0.0.4; charset=utf-8')
    self.send_header('Cache-Control', 'no-store')
    self.send_header('Content-Length', str(len(body)))
    self.end_headers()
    self.wfile.write(body)
```

7.7 L55 クラス DashboardNode

- 行範囲: L55–L406
- 所属: module 直下
- 定義: `DashboardNode(bases=Node)`
- `docstring`: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: 特に明示されていない。
- 戻り値の要点: 戻り値が無いか、`return` 文が明示されていない。
- 主なローカル代入名: 特になし。
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、`try` 0

この定義を読むための Python 構文

- `class` 文は新しい型を定義する。丸括弧内は継承する基底クラスである。
- 内包表記は、反復・条件・値生成を一つの式で記述して `collection` を作る。
- `with` 文は `context manager` に開始・終了処理を任せ、ファイルなどの資源を確実に解放する。
- `try` 文は例外が起き得る処理と、異常時または終了時の経路を分ける。

上から順の処理

1. 関数 `__init__` を定義する。
2. 関数 `destroy_node` を定義する。
3. 関数 `get_state` を定義する。
4. 関数 `get_prometheus_metrics` を定義する。
5. 関数 `_update` を定義する。
6. 関数 `_on_speed_cmd` を定義する。
7. 関数 `_on_upper_speed_cmd` を定義する。
8. 関数 `_on_speed_meas` を定義する。
9. 関数 `_on_throttle_cmd` を定義する。
10. 関数 `_on_drive_mode` を定義する。
11. 関数 `_on_soc` を定義する。
12. 関数 `_set_estimated_soc` を定義する。

代表コード断片

```
class DashboardNode(Node):
    def __init__(self):
        super().__init__('dashboard_node')
        self.declare_parameter('host', '0.0.0.0')
        self.declare_parameter('port', 8080)
        self.declare_parameter('static_dir', '')
        self.declare_parameter('dummy_csv', '')
        self.declare_parameter('dummy_rate_hz', 5.0)

        static_dir = str(self.get_parameter('static_dir').value).strip()
        if not static_dir:
            try:
                pkg_share = get_package_share_directory('mpc_solarcar')
                static_dir = os.path.join(pkg_share, 'dashboard')
            except Exception:
                static_dir = os.path.join(os.path.dirname(__file__), '..', 'dashboard')
        static_dir = os.path.abspath(static_dir)

        self._lock = threading.Lock()
        self._state = {
            'ts': time.time(),
            'speed_cmd_kmh': None,
            'upper_speed_cmd_kmh': None,
            'speed_meas_kmh': None,
            'model_speed_kmh': None,
            'throttle_cmd_pct': None,
            'drive_mode': None,
            'soc': None,
            'soc_meas': None,
            'soc_est': None,
            'Tb_C': None,
            's_kmh': None,
            'batt_current_a': None,
            'batt_voltage_v': None,
            'motor_w': None,
            ...
```

7.8 L56 関数 DashboardNode.__init__

- 行範囲: L56–L154
- 所属: DashboardNode
- 定義: `__init__(self)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `Lock`, `Thread`, `ThreadingHTTPServer`, `__init__`, `_init_dummy`, `abspath`, `create_subscription`, `declare_parameter`, `dirname`, `error`, `float`, `get_logger`
- 戻り値の要点: 戻り値が無いか、`return` 文が明示されていない。
- 主なローカル代入名: `dummy_csv`, `handler`, `host`, `pkg_share`, `port`, `static_dir`
- 読み取る主なインスタンス属性: `self._init_dummy`, `self._on_drive_mode`, `self._on_env`, `self._on_gps`, `self._on_health`, `self._on_ibatt`, `self._on_lower_plan`, `self._on_metrics`, `self._on_mpc_state`, `self._on_path`, `self._on_s_km`, `self._on_soc`, `self._on_speed_cmd`, `self._on_speed_meas`, `self._on_status`, `self._on_sys_diag`, `self._on_sys_state`, `self._on_tb`, `self._on_throttle_cmd`, `self._on_upper_plan`, `self._on_upper_speed_cmd`, `self._on_vbatt`, `self._server`, `self._server_thread`
- 更新する主なインスタンス属性: `self._lock`, `self._server`, `self._server_thread`, `self._soc_meas_monotonic`, `self._state`
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 2、ループ 0、`try` 2

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。
- `try` 文は例外が起き得る処理と、異常時または終了時の経路を分ける。

上から順の処理

1. `super().__init__(...)` を実行する。
2. `self.declare_parameter(...)` を実行する。
3. `self.declare_parameter(...)` を実行する。
4. `self.declare_parameter(...)` を実行する。
5. `self.declare_parameter(...)` を実行する。
6. `self.declare_parameter(...)` を実行する。
7. `static_dir` に `str(self.get_parameter('static_dir').value).strip()` の結果を代入する。
8. 条件 `not static_dir` を判定し、真なら内部処理を行う。
9. 例外処理を伴う `try` ブロックを実行する。
10. `pkg_share` に `get_package_share_directory('mpc_solarcar')` の結果を代入する。
11. `static_dir` に `os.path.join(pkg_share, 'dashboard')` の結果を代入する。
12. `Exception` を捕捉した場合:
13. `static_dir` に `os.path.join(os.path.dirname(__file__), '..', 'dashboard')` の結果を代入する。

14. `static_dir` に `os.path.abspath(static_dir)` の結果を代入する。
15. `self._lock` に `threading.Lock()` の結果を代入する。
16. `self._state` に `{'ts': time.time(), 'speed_cmd_kmh': None, 'upper_speed_cmd_kmh': None, 'speed_meas_kmh': None, 'model_speed_kmh': None, 'throttle_cmd_pct': None, 'drive_mode': None, 'soc': None, 'soc_meas': None, 'soc_est': None, 'Tb_C': None, 's_kmh': None, 'batt_current_a': None, 'batt_voltage_v': None, 'motor_w': None, 'motor_a': None, 'solar_w': None, 'wheel_w': None, 'pack_w': None, 'G_poa': None, 'Tcell_C': None, 'Tamb_C': None, 'headwind_ms': None, 'slope_pct': None, 'plan_dt': None, 'lower_dt': None, 'plan_upper': None, 'plan_lower': None, 'forecast_k': None, 'sec_to_next': None, 'control_stop_hold': None, 'control_stop_remaining_sec': None, 'control_stop_completed_count': None, 'system_state': None, 'system_diag': None, 'mpc_state': None, 'system_health': None, 'gps_lat': None, 'gps_lon': None}` の結果を代入する。
17. `self._soc_meas_monotonic` に `None` の結果を代入する。
18. `self.create_subscription(...)` を実行する。
19. `self.create_subscription(...)` を実行する。
20. `self.create_subscription(...)` を実行する。
21. `self.create_subscription(...)` を実行する。
22. `self.create_subscription(...)` を実行する。
23. `self.create_subscription(...)` を実行する。
24. `self.create_subscription(...)` を実行する。
25. `self.create_subscription(...)` を実行する。
26. `self.create_subscription(...)` を実行する。
27. `self.create_subscription(...)` を実行する。
28. `self.create_subscription(...)` を実行する。
29. `self.create_subscription(...)` を実行する。
30. `self.create_subscription(...)` を実行する。
31. `self.create_subscription(...)` を実行する。
32. `self.create_subscription(...)` を実行する。
33. `self.create_subscription(...)` を実行する。
34. `self.create_subscription(...)` を実行する。
35. `self.create_subscription(...)` を実行する。
36. `self.create_subscription(...)` を実行する。
37. `self.create_subscription(...)` を実行する。
38. `self.create_subscription(...)` を実行する。
39. `dummy_csv` に `str(self.get_parameter('dummy_csv').value).strip()` の結果を代入する。
40. 条件 `dummy_csv` を判定し、真なら内部処理を行う。
41. `self._init_dummy(...)` を実行する。
42. `self._server` に `None` の結果を代入する。

43. self._server_thread に None の結果を代入する。
44. host に str(self.get_parameter('host').value) の結果を代入する。
45. port に int(self.get_parameter('port').value) の結果を代入する。
46. handler に partial(DashboardHandler, node=self, directory=static_dir) の結果を代入する。
47. 例外処理を伴う try ブロックを実行する。
48. self._server に ThreadingHTTPServer((host, port), handler) の結果を代入する。
49. self._server_thread に threading.Thread(target=self._server.serve_forever, daemon=True) の結果を代入する。
50. self._server_thread.start(...) を実行する。
51. self.get_logger().info(...) を実行する。
52. Exception を捕捉した場合:
53. self.get_logger().error(...) を実行する。

代表コード断片

```
def __init__(self):
    super().__init__('dashboard_node')
    self.declare_parameter('host', '0.0.0.0')
    self.declare_parameter('port', 8080)
    self.declare_parameter('static_dir', '')
    self.declare_parameter('dummy_csv', '')
    self.declare_parameter('dummy_rate_hz', 5.0)

    static_dir = str(self.get_parameter('static_dir').value).strip()
    if not static_dir:
        try:
            pkg_share = get_package_share_directory('mpc_solarcar')
            static_dir = os.path.join(pkg_share, 'dashboard')
        except Exception:
            static_dir = os.path.join(os.path.dirname(__file__), '..', 'dashboard')
    static_dir = os.path.abspath(static_dir)

    self._lock = threading.Lock()
    self._state = {
        'ts': time.time(),
        'speed_cmd_kmh': None,
        'upper_speed_cmd_kmh': None,
        'speed_meas_kmh': None,
        'model_speed_kmh': None,
        'throttle_cmd_pct': None,
        'drive_mode': None,
        'soc': None,
        'soc_meas': None,
        'soc_est': None,
        'Tb_C': None,
        's_km': None,
        'batt_current_a': None,
        'batt_voltage_v': None,
        'motor_w': None,
        'motor_a': None,
        ...
```

7.9 L156 関数 DashboardNode.destroy_node

- 行範囲: L156–L163
- 所属: DashboardNode
- 定義: `destroy_node(self)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `destroy_node`, `server_close`, `shutdown`, `super`
- 戻り値の要点: `super().destroy_node()`
- 主なローカル代入名: 特になし。
- 読み取る主なインスタンス属性: `self._server`
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 1、ループ 0、try 1

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。
- `try` 文は例外が起き得る処理と、異常時または終了時の経路を分ける。

上から順の処理

1. 条件 `self._server is not None` を判定し、真なら内部処理を行う。
2. 例外処理を伴う `try` ブロックを実行する。
3. `self._server.shutdown(...)` を実行する。
4. `self._server.server_close(...)` を実行する。
5. `Exception` を捕捉した場合:
6. `Pass` 文を実行する。
7. `super().destroy_node()` を返す。

代表コード断片

```
def destroy_node(self):
    if self._server is not None:
        try:
            self._server.shutdown()
            self._server.server_close()
        except Exception:
            pass
    return super().destroy_node()
```

7.10 L165 関数 DashboardNode.get_state

- 行範囲: L165–L167
- 所属: DashboardNode
- 定義: `get_state(self)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `dict`
- 戻り値の要点: `dict(self._state)`
- 主なローカル代入名: 特になし。
- 読み取る主なインスタンス属性: `self._lock`, `self._state`
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、try 0

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。
- `with` 文は `context manager` に開始・終了処理を任せ、ファイルなどの資源を確実に解放する。

上から順の処理

1. `with` 文で `self._lock` を管理しながら処理する。
2. `dict(self._state)` を返す。

代表コード断片

```
def get_state(self):  
    with self._lock:  
        return dict(self._state)
```

7.11 L169 関数 DashboardNode.get_prometheus_metrics

- 行範囲: L169–L234
- 所属: DashboardNode
- 定義: `get_prometheus_metrics(self)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `dict`, `extend`, `float`, `get`, `isfinite`, `items`, `join`, `max`, `replace`, `str`, `time`
- 戻り値の要点: `'\n'.join(lines) + '\n'`
- 主なローカル代入名: `age_sec`, `help_text`, `key`, `label`, `lines`, `metric_keys`, `metric_name`, `number`, `state`, `state_key`, `value`
- 読み取る主なインスタンス属性: `self._lock`, `self._state`
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 1、ループ 2、try 1

この定義を読むための Python 構文

- 第 1 引数 self は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。
- with 文は context manager に開始・終了処理を任せ、ファイルなどの資源を確実に解放する。
- try 文は例外が起き得る処理と、異常時または終了時の経路を分ける。

上から順の処理

1. metric_keys に {'speed_cmd_kmh': ('solarcar_speed_command_kmh', 'Lower speed command'), 'upper_speed_cmd_kmh': ('solarcar_speed_upper_command_kmh', 'Upper planner speed command'), 'speed_meas_kmh': ('solarcar_speed_measured_kmh', 'Measured vehicle speed'), 'model_speed_kmh': ('solarcar_speed_model_kmh', 'Speed used for planner model metrics'), 'throttle_cmd_pct': ('solarcar_throttle_command_percent', 'Lower controller throttle command'), 'soc': ('solarcar_battery_soc_ratio', 'Selected battery state of charge'), 'soc_meas': ('solarcar_battery_soc_measured_ratio', 'Measured battery state of charge'), 'soc_est': ('solarcar_battery_soc_estimated_ratio', 'Estimated battery state of charge'), 'Tb_C': ('solarcar_battery_temperature_celsius', 'Battery temperature'), 's_km': ('solarcar_distance_km', 'Race distance'), 'batt_current_a': ('solarcar_battery_current_ampere', 'Battery current'), 'batt_voltage_v': ('solarcar_battery_voltage_volt', 'Battery terminal voltage'), 'motor_w': ('solarcar_motor_power_watt', 'Motor electrical power'), 'motor_a': ('solarcar_motor_current_ampere', 'Motor current'), 'solar_w': ('solarcar_pv_power_watt', 'PV power'), 'wheel_w': ('solarcar_wheel_power_watt', 'Wheel mechanical power'), 'pack_w': ('solarcar_pack_power_watt', 'Battery pack power'), 'G_poa': ('solarcar_irradiance_poa_watt_per_m2', 'Plane-of-array irradiance'), 'Tcell_C': ('solarcar_cell_temperature_celsius', 'PV cell temperature'), 'Tamb_C': ('solarcar_ambient_temperature_celsius', 'Ambient temperature'), 'headwind_ms': ('solarcar_headwind_meter_per_second', 'Corrected headwind'), 'slope_pct': ('solarcar_route_slope_percent', 'Route slope'), 'forecast_k': ('solarcar_forecast_index', 'Forecast row index'), 'sec_to_next': ('solarcar_seconds_to_next_update', 'Seconds to next planner update'), 'control_stop_hold': ('solarcar_control_stop_hold', 'Control-stop approach or dwell is active'), 'control_stop_remaining_sec': ('solarcar_control_stop_remaining_seconds', 'Remaining mandatory control-stop dwell'), 'control_stop_completed_count': ('solarcar_control_stop_completed_total', 'Completed mandatory control stops'), 'system_health': ('solarcar_system_health_ratio', 'System health ratio'), 'gps_lat': ('solarcar_gps_latitude_degree', 'GNSS latitude'), 'gps_lon': ('solarcar_gps_longitude_degree', 'GNSS longitude'), 'path_points': ('solarcar_path_points', 'Published trajectory point count')} の結果を代入する。
2. with 文で self._lock を管理しながら処理する。
3. state に dict(self._state) の結果を代入する。
4. lines に [] の結果を代入する。
5. metric_keys.items() を順に走査し、各要素を (state_key, (metric_name, help_text)) に入れて処理する。
6. value に state.get(state_key) の結果を代入する。
7. 例外処理を伴う try ブロックを実行する。
8. number に float(value) の結果を代入する。
9. (TypeError, ValueError) を捕捉した場合:
10. Continue 文を実行する。
11. 条件 not math.isfinite(number) を判定し、真なら内部処理を行う。
12. Continue 文を実行する。
13. lines.extend(...) を実行する。
14. age_sec に max(0.0, time.time() - float(state.get('ts', time.time()))) の結果を代入する。

15. `lines.extend(...)` を実行する。
16. `(('system_state', 'solarcar_system_state_info'), ('mpc_state', 'solarcar_mpc_state_info'), ('drive_mode', 'solarcar_drive_mode_info'))` を順に走査し、各要素を `(key, metric_name)` に入れて処理する。
17. `label` に `str(state.get(key) or 'unknown').replace('\n', '\\n').replace('\"', '\\\"').replace('\n', ' ')` の結果を代入する。
18. `lines.extend(...)` を実行する。
19. `'\n'.join(lines) + '\n'` を返す。

代表コード断片

```
def get_prometheus_metrics(self):
    metric_keys = {
        'speed_cmd_kmh': ('solarcar_speed_command_kmh', 'Lower speed command'),
        'upper_speed_cmd_kmh': ('solarcar_speed_upper_command_kmh', 'Upper planner speed
command'),
        'speed_meas_kmh': ('solarcar_speed_measured_kmh', 'Measured vehicle speed'),
        'model_speed_kmh': ('solarcar_speed_model_kmh', 'Speed used for planner model
metrics'),
        'throttle_cmd_pct': ('solarcar_throttle_command_percent', 'Lower controller
throttle command'),
        'soc': ('solarcar_battery_soc_ratio', 'Selected battery state of charge'),
        'soc_meas': ('solarcar_battery_soc_measured_ratio', 'Measured battery state of
charge'),
        'soc_est': ('solarcar_battery_soc_estimated_ratio', 'Estimated battery state of
charge'),
        'Tb_C': ('solarcar_battery_temperature_celsius', 'Battery temperature'),
        's_km': ('solarcar_distance_km', 'Race distance'),
        'batt_current_a': ('solarcar_battery_current_ampere', 'Battery current'),
        'batt_voltage_v': ('solarcar_battery_voltage_volt', 'Battery terminal voltage'),
        'motor_w': ('solarcar_motor_power_watt', 'Motor electrical power'),
        'motor_a': ('solarcar_motor_current_ampere', 'Motor current'),
        'solar_w': ('solarcar_pv_power_watt', 'PV power'),
        'wheel_w': ('solarcar_wheel_power_watt', 'Wheel mechanical power'),
        'pack_w': ('solarcar_pack_power_watt', 'Battery pack power'),
        'G_poa': ('solarcar_irradiance_poa_watt_per_m2', 'Plane-of-array irradiance'),
        'Tcell_C': ('solarcar_cell_temperature_celsius', 'PV cell temperature'),
        'Tamb_C': ('solarcar_ambient_temperature_celsius', 'Ambient temperature'),
        'headwind_ms': ('solarcar_headwind_meter_per_second', 'Corrected headwind'),
        'slope_pct': ('solarcar_route_slope_percent', 'Route slope'),
        'forecast_k': ('solarcar_forecast_index', 'Forecast row index'),
        'sec_to_next': ('solarcar_seconds_to_next_update', 'Seconds to next planner
update'),
        'control_stop_hold': ('solarcar_control_stop_hold', 'Control-stop approach or
dwell is active'),
        'control_stop_remaining_sec': ('solarcar_control_stop_remaining_seconds', '
Remaining mandatory control-stop dwell'),
        'control_stop_completed_count': ('solarcar_control_stop_completed_total', '
Completed mandatory control stops'),
        'system_health': ('solarcar_system_health_ratio', 'System health ratio'),
        'gps_lat': ('solarcar_gps_latitude_degree', 'GNSS latitude'),
        'gps_lon': ('solarcar_gps_longitude_degree', 'GNSS longitude'),
        'path_points': ('solarcar_path_points', 'Published trajectory point count'),
    }
    with self._lock:
        ...
```

7.12 L236 関数 `DashboardNode._update`

- 行範囲: L236–L239

- 所属: DashboardNode
- 定義: `_update(self, key, value)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `time`
- 戻り値の要点: 戻り値が無いか、`return` 文が明示されていない。
- 主なローカル代入名: 特になし。
- 読み取る主なインスタンス属性: `self._lock`, `self._state`
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、`try` 0

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。
- `with` 文は `context manager` に開始・終了処理を任せ、ファイルなどの資源を確実に解放する。

上から順の処理

1. `with` 文で `self._lock` を管理しながら処理する。
2. `self._state[key]` に `value` の結果を代入する。
3. `self._state['ts']` に `time.time()` の結果を代入する。

代表コード断片

```
def _update(self, key, value):
    with self._lock:
        self._state[key] = value
        self._state['ts'] = time.time()
```

7.13 L241 関数 DashboardNode._on_speed_cmd

- 行範囲: L241–L242
- 所属: DashboardNode
- 定義: `_on_speed_cmd(self, msg: Float32)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `_update`, `float`
- 戻り値の要点: 戻り値が無いか、`return` 文が明示されていない。
- 主なローカル代入名: 特になし。
- 読み取る主なインスタンス属性: `self._update`
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、`try` 0

この定義を読むための Python 構文

- 第 1 引数 self は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。

上から順の処理

1. self._update(...) を実行する。

代表コード断片

```
def _on_speed_cmd(self, msg: Float32):  
    self._update('speed_cmd_kmh', float(msg.data))
```

7.14 L244 関数 DashboardNode._on_upper_speed_cmd

- 行範囲: L244–L245
- 所属: DashboardNode
- 定義: _on_upper_speed_cmd(self, msg: Float32)
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: _update, float
- 戻り値の要点: 戻り値が無いか、return 文が明示されていない。
- 主なローカル代入名: 特になし。
- 読み取る主なインスタンス属性: self._update
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 raise はない。
- 制御構造の規模: 条件分岐 0、ループ 0、try 0

この定義を読むための Python 構文

- 第 1 引数 self は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。

上から順の処理

1. self._update(...) を実行する。

代表コード断片

```
def _on_upper_speed_cmd(self, msg: Float32):  
    self._update('upper_speed_cmd_kmh', float(msg.data))
```

7.15 L247 関数 DashboardNode._on_speed_meas

- 行範囲: L247–L248
- 所属: DashboardNode
- 定義: _on_speed_meas(self, msg: Float32)
- docstring: 明示されていない。

- このブロックが直接呼ぶ主な関数・メソッド: `_update`, `float`
- 戻り値の要点: 戻り値が無いか、`return` 文が明示されていない。
- 主なローカル代入名: 特になし。
- 読み取る主なインスタンス属性: `self._update`
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、`try` 0

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。

上から順の処理

1. `self._update(...)` を実行する。

代表コード断片

```
def _on_speed_meas(self, msg: Float32):
    self._update('speed_meas_kmh', float(msg.data))
```

7.16 L250 関数 `DashboardNode._on_throttle_cmd`

- 行範囲: L250–L251
- 所属: `DashboardNode`
- 定義: `_on_throttle_cmd(self, msg: Float32)`
- `docstring`: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `_update`, `float`
- 戻り値の要点: 戻り値が無いか、`return` 文が明示されていない。
- 主なローカル代入名: 特になし。
- 読み取る主なインスタンス属性: `self._update`
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、`try` 0

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。

上から順の処理

1. `self._update(...)` を実行する。

代表コード断片

```
def _on_throttle_cmd(self, msg: Float32):
    self._update('throttle_cmd_pct', float(msg.data))
```

7.17 L253 関数 DashboardNode._on_drive_mode

- 行範囲: L253–L254
- 所属: DashboardNode
- 定義: `_on_drive_mode(self, msg: String)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `_update`, `str`
- 戻り値の要点: 戻り値が無いか、`return` 文が明示されていない。
- 主なローカル代入名: 特になし。
- 読み取る主なインスタンス属性: `self._update`
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、`try` 0

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。

上から順の処理

1. `self._update(...)` を実行する。

代表コード断片

```
def _on_drive_mode(self, msg: String):
    self._update('drive_mode', str(msg.data))
```

7.18 L256 関数 DashboardNode._on_soc

- 行範囲: L256–L262
- 所属: DashboardNode
- 定義: `_on_soc(self, msg: Float32)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `float`, `monotonic`, `time`
- 戻り値の要点: 戻り値が無いか、`return` 文が明示されていない。
- 主なローカル代入名: `value`
- 読み取る主なインスタンス属性: `self._lock`, `self._state`
- 更新する主なインスタンス属性: `self._soc_meas_monotonic`
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、`try` 0

この定義を読むための Python 構文

- 第 1 引数 self は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。
- with 文は context manager に開始・終了処理を任せ、ファイルなどの資源を確実に解放する。

上から順の処理

1. value に float(msg.data) の結果を代入する。
2. with 文で self._lock を管理しながら処理する。
3. self._state['soc'] に value の結果を代入する。
4. self._state['soc_meas'] に value の結果を代入する。
5. self._state['ts'] に time.time() の結果を代入する。
6. self._soc_meas_monotonic に time.monotonic() の結果を代入する。

代表コード断片

```
def _on_soc(self, msg: Float32):  
    value = float(msg.data)  
    with self._lock:  
        self._state['soc'] = value  
        self._state['soc_meas'] = value  
        self._state['ts'] = time.time()  
        self._soc_meas_monotonic = time.monotonic()
```

7.19 L264 関数 DashboardNode._set_estimated_soc

- 行範囲: L264–L271
- 所属: DashboardNode
- 定義: _set_estimated_soc(self, value)
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: float, monotonic
- 戻り値の要点: 戻り値が無い、return 文が明示されていない。
- 主なローカル代入名: measured_is_fresh
- 読み取る主なインスタンス属性: self._soc_meas_monotonic, self._state
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 raise はない。
- 制御構造の規模: 条件分岐 1、ループ 0、try 0

この定義を読むための Python 構文

- 第 1 引数 self は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。

上から順の処理

1. `self._state['soc_est']` に `float(value)` の結果を代入する。
2. `measured_is_fresh` に `self._soc_meas_monotonic is not None and time.monotonic() - self._soc_meas_monotonic <= 5.0` の結果を代入する。
3. 条件 `not measured_is_fresh` を判定し、真なら内部処理を行う。
4. `self._state['soc']` に `float(value)` の結果を代入する。

代表コード断片

```
def _set_estimated_soc(self, value):
    self._state['soc_est'] = float(value)
    measured_is_fresh = (
        self._soc_meas_monotonic is not None
        and (time.monotonic() - self._soc_meas_monotonic) <= 5.0
    )
    if not measured_is_fresh:
        self._state['soc'] = float(value)
```

7.20 L273 関数 `DashboardNode._on_tb`

- 行範囲: L273–L274
- 所属: `DashboardNode`
- 定義: `_on_tb(self, msg: Float32)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `_update`, `float`
- 戻り値の要点: 戻り値が無いか、`return` 文が明示されていない。
- 主なローカル代入名: 特になし。
- 読み取る主なインスタンス属性: `self._update`
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、`try` 0

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。

上から順の処理

1. `self._update(...)` を実行する。

代表コード断片

```
def _on_tb(self, msg: Float32):
    self._update('Tb_C', float(msg.data))
```

7.21 L276 関数 DashboardNode._on_ibatt

- 行範囲: L276–L277
- 所属: DashboardNode
- 定義: `_on_ibatt(self, msg: Float32)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `_update`, `float`
- 戻り値の要点: 戻り値が無いか、`return` 文が明示されていない。
- 主なローカル代入名: 特になし。
- 読み取る主なインスタンス属性: `self._update`
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、`try` 0

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。

上から順の処理

1. `self._update(...)` を実行する。

代表コード断片

```
def _on_ibatt(self, msg: Float32):  
    self._update('batt_current_a', float(msg.data))
```

7.22 L279 関数 DashboardNode._on_vbatt

- 行範囲: L279–L280
- 所属: DashboardNode
- 定義: `_on_vbatt(self, msg: Float32)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `_update`, `float`
- 戻り値の要点: 戻り値が無いか、`return` 文が明示されていない。
- 主なローカル代入名: 特になし。
- 読み取る主なインスタンス属性: `self._update`
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、`try` 0

この定義を読むための Python 構文

- 第 1 引数 self は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。

上から順の処理

1. self._update(...) を実行する。

代表コード断片

```
def _on_vbatt(self, msg: Float32):  
    self._update('batt_voltage_v', float(msg.data))
```

7.23 L282 関数 DashboardNode._on_s_km

- 行範囲: L282–L283
- 所属: DashboardNode
- 定義: _on_s_km(self, msg: Float32)
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: _update, float
- 戻り値の要点: 戻り値が無いか、return 文が明示されていない。
- 主なローカル代入名: 特になし。
- 読み取る主なインスタンス属性: self._update
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 raise はない。
- 制御構造の規模: 条件分岐 0、ループ 0、try 0

この定義を読むための Python 構文

- 第 1 引数 self は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。

上から順の処理

1. self._update(...) を実行する。

代表コード断片

```
def _on_s_km(self, msg: Float32):  
    self._update('s_km', float(msg.data))
```

7.24 L285 関数 DashboardNode._on_status

- 行範囲: L285–L299
- 所属: DashboardNode
- 定義: _on_status(self, msg: Float32MultiArray)
- docstring: 明示されていない。

- このブロックが直接呼ぶ主な関数・メソッド: `_set_estimated_soc`, `float`, `len`, `list`, `time`
- 戻り値の要点: 戻り値が無い、`return` 文が明示されていない。
- 主なローカル代入名: `data`
- 読み取る主なインスタンス属性: `self._lock`, `self._set_estimated_soc`, `self._state`
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 2、ループ 0、`try` 0

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。
- `with` 文は `context manager` に開始・終了処理を任せ、ファイルなどの資源を確実に解放する。

上から順の処理

1. `data` に `list(msg.data)` の結果を代入する。
2. `with` 文で `self._lock` を管理しながら処理する。
3. 条件 `len(data) >= 5` を判定し、真なら内部処理を行う。
4. `self._set_estimated_soc(...)` を実行する。
5. `self._state['Tb_C']` に `float(data[1])` の結果を代入する。
6. `self._state['s_km']` に `float(data[2])` の結果を代入する。
7. `self._state['forecast_k']` に `float(data[3])` の結果を代入する。
8. `self._state['sec_to_next']` に `float(data[4])` の結果を代入する。
9. 条件 `len(data) >= 8` を判定し、真なら内部処理を行う。
10. `self._state['control_stop_hold']` に `float(data[5])` の結果を代入する。
11. `self._state['control_stop_remaining_sec']` に `float(data[6])` の結果を代入する。
12. `self._state['control_stop_completed_count']` に `float(data[7])` の結果を代入する。
13. `self._state['status_raw']` に `data` の結果を代入する。
14. `self._state['ts']` に `time.time()` の結果を代入する。

代表コード断片

```
def _on_status(self, msg: Float32MultiArray):
    data = list(msg.data)
    with self._lock:
        if len(data) >= 5:
            self._set_estimated_soc(data[0])
            self._state['Tb_C'] = float(data[1])
            self._state['s_km'] = float(data[2])
            self._state['forecast_k'] = float(data[3])
            self._state['sec_to_next'] = float(data[4])
        if len(data) >= 8:
            self._state['control_stop_hold'] = float(data[5])
            self._state['control_stop_remaining_sec'] = float(data[6])
            self._state['control_stop_completed_count'] = float(data[7])
    self._state['status_raw'] = data
    self._state['ts'] = time.time()
```

7.25 L301 関数 DashboardNode._on_env

- 行範囲: L301–L311
- 所属: DashboardNode
- 定義: `_on_env(self, msg: Float32MultiArray)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `float`, `len`, `list`, `time`
- 戻り値の要点: 戻り値が無いが、`return` 文が明示されていない。
- 主なローカル代入名: `data`
- 読み取る主なインスタンス属性: `self._lock`, `self._state`
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 1、ループ 0、`try` 0

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。
- `with` 文は `context manager` に開始・終了処理を任せ、ファイルなどの資源を確実に解放する。

上から順の処理

1. `data` に `list(msg.data)` の結果を代入する。
2. 条件 `len(data) < 5` を判定し、真なら内部処理を行う。
3. を返す。
4. `with` 文で `self._lock` を管理しながら処理する。
5. `self._state['G_poa']` に `float(data[0])` の結果を代入する。
6. `self._state['Tcell_C']` に `float(data[1])` の結果を代入する。
7. `self._state['Tamb_C']` に `float(data[2])` の結果を代入する。
8. `self._state['slope_pct']` に `float(data[3])` の結果を代入する。
9. `self._state['headwind_ms']` に `float(data[4])` の結果を代入する。
10. `self._state['ts']` に `time.time()` の結果を代入する。

代表コード断片

```
def _on_env(self, msg: Float32MultiArray):
    data = list(msg.data)
    if len(data) < 5:
        return
    with self._lock:
        self._state['G_poa'] = float(data[0])
        self._state['Tcell_C'] = float(data[1])
        self._state['Tamb_C'] = float(data[2])
        self._state['slope_pct'] = float(data[3])
        self._state['headwind_ms'] = float(data[4])
        self._state['ts'] = time.time()
```

7.26 L313 関数 DashboardNode._on_metrics

- 行範囲: L313–L331
- 所属: DashboardNode
- 定義: `_on_metrics(self, msg: Float32MultiArray)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `_set_estimated_soc`, `float`, `len`, `list`, `time`
- 戻り値の要点: 戻り値が無いか、`return` 文が明示されていない。
- 主なローカル代入名: `data`
- 読み取る主なインスタンス属性: `self._lock`, `self._set_estimated_soc`, `self._state`
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 2、ループ 0、`try` 0

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。
- `with` 文は `context manager` に開始・終了処理を任せ、ファイルなどの資源を確実に解放する。

上から順の処理

1. `data` に `list(msg.data)` の結果を代入する。
2. 条件 `len(data) < 8` を判定し、真なら内部処理を行う。
3. を返す。
4. `with` 文で `self._lock` を管理しながら処理する。
5. `self._state['batt_voltage_v']` に `float(data[0])` の結果を代入する。
6. `self._state['batt_current_a']` に `float(data[1])` の結果を代入する。
7. `self._set_estimated_soc(...)` を実行する。
8. `self._state['motor_w']` に `float(data[3])` の結果を代入する。
9. `self._state['motor_a']` に `float(data[4])` の結果を代入する。
10. `self._state['solar_w']` に `float(data[5])` の結果を代入する。
11. `self._state['model_speed_kmh']` に `float(data[6])` の結果を代入する。
12. `self._state['wheel_w']` に `float(data[7])` の結果を代入する。
13. 条件 `len(data) >= 9` を判定し、真なら内部処理を行う。
14. `self._state['pack_w']` に `float(data[8])` の結果を代入する。
15. `self._state['ts']` に `time.time()` の結果を代入する。

代表コード断片

```
def _on_metrics(self, msg: Float32MultiArray):
    data = list(msg.data)
    if len(data) < 8:
        return
    with self._lock:
        self._state['batt_voltage_v'] = float(data[0])
        self._state['batt_current_a'] = float(data[1])
        self._set_estimated_soc(data[2])
        self._state['motor_w'] = float(data[3])
        self._state['motor_a'] = float(data[4])
        self._state['solar_w'] = float(data[5])
        # Metrics carry the speed used for model evaluation, not the lower
        # controller command. Keep it separate so callback ordering cannot
        # overwrite /planner/speed_cmd on the dashboard.
        self._state['model_speed_kmh'] = float(data[6])
        self._state['wheel_w'] = float(data[7])
        if len(data) >= 9:
            self._state['pack_w'] = float(data[8])
        self._state['ts'] = time.time()
```

7.27 L333 関数 DashboardNode._on_upper_plan

- 行範囲: L333–L342
- 所属: DashboardNode
- 定義: `_on_upper_plan(self, msg: Float32MultiArray)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `float`, `len`, `list`, `time`
- 戻り値の要点: 戻り値が無いか、`return` 文が明示されていない。
- 主なローカル代入名: `data`, `dt`, `speeds`, `v`
- 読み取る主なインスタンス属性: `self._lock`, `self._state`
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 1、ループ 0、`try` 0

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。
- 内包表記は、反復・条件・値生成を一つの式で記述して `collection` を作る。
- `with` 文は `context manager` に開始・終了処理を任せ、ファイルなどの資源を確実に解放する。

上から順の処理

1. `data` に `list(msg.data)` の結果を代入する。
2. 条件 `len(data) < 2` を判定し、真なら内部処理を行う。
3. を返す。

4. `dt` に `float(data[0])` の結果を代入する。
5. `speeds` に `[float(v) for v in data[1:]]` の結果を代入する。
6. `with` 文で `self._lock` を管理しながら処理する。
7. `self._state['plan_dt']` に `dt` の結果を代入する。
8. `self._state['plan_upper']` に `speeds` の結果を代入する。
9. `self._state['ts']` に `time.time()` の結果を代入する。

代表コード断片

```
def _on_upper_plan(self, msg: Float32MultiArray):
    data = list(msg.data)
    if len(data) < 2:
        return
    dt = float(data[0])
    speeds = [float(v) for v in data[1:]]
    with self._lock:
        self._state['plan_dt'] = dt
        self._state['plan_upper'] = speeds
        self._state['ts'] = time.time()
```

7.28 L344 関数 `DashboardNode._on_lower_plan`

- 行範囲: L344–L353
- 所属: `DashboardNode`
- 定義: `_on_lower_plan(self, msg: Float32MultiArray)`
- `docstring`: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `float`, `len`, `list`, `time`
- 戻り値の要点: 戻り値が無いか、`return` 文が明示されていない。
- 主なローカル代入名: `data`, `dt`, `speeds`, `v`
- 読み取る主なインスタンス属性: `self._lock`, `self._state`
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 1、ループ 0、`try` 0

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。
- 内包表記は、反復・条件・値生成を一つの式で記述して `collection` を作る。
- `with` 文は `context manager` に開始・終了処理を任せ、ファイルなどの資源を確実に解放する。

上から順の処理

1. data に `list(msg.data)` の結果を代入する。
2. 条件 `len(data) < 2` を判定し、真なら内部処理を行う。
3. を返す。
4. dt に `float(data[0])` の結果を代入する。
5. speeds に `[float(v) for v in data[1:]]` の結果を代入する。
6. with 文で `self._lock` を管理しながら処理する。
7. `self._state['lower_dt']` に dt の結果を代入する。
8. `self._state['plan_lower']` に speeds の結果を代入する。
9. `self._state['ts']` に `time.time()` の結果を代入する。

代表コード断片

```
def _on_lower_plan(self, msg: Float32MultiArray):
    data = list(msg.data)
    if len(data) < 2:
        return
    dt = float(data[0])
    speeds = [float(v) for v in data[1:]]
    with self._lock:
        self._state['lower_dt'] = dt
        self._state['plan_lower'] = speeds
        self._state['ts'] = time.time()
```

7.29 L355 関数 `DashboardNode._on_sys_state`

- 行範囲: L355–L356
- 所属: `DashboardNode`
- 定義: `_on_sys_state(self, msg: String)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `_update`, `str`
- 戻り値の要点: 戻り値が無いが、`return` 文が明示されていない。
- 主なローカル代入名: 特になし。
- 読み取る主なインスタンス属性: `self._update`
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、`try` 0

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。

上から順の処理

1. `self._update(...)` を実行する。

代表コード断片

```
def _on_sys_state(self, msg: String):  
    self._update('system_state', str(msg.data))
```

7.30 L358 関数 `DashboardNode._on_sys_diag`

- 行範囲: L358–L359
- 所属: `DashboardNode`
- 定義: `_on_sys_diag(self, msg: String)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `_update`, `str`
- 戻り値の要点: 戻り値が無いか、`return` 文が明示されていない。
- 主なローカル代入名: 特になし。
- 読み取る主なインスタンス属性: `self._update`
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、`try` 0

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。

上から順の処理

1. `self._update(...)` を実行する。

代表コード断片

```
def _on_sys_diag(self, msg: String):  
    self._update('system_diag', str(msg.data))
```

7.31 L361 関数 `DashboardNode._on_mpc_state`

- 行範囲: L361–L362
- 所属: `DashboardNode`
- 定義: `_on_mpc_state(self, msg: String)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `_update`, `str`
- 戻り値の要点: 戻り値が無いか、`return` 文が明示されていない。
- 主なローカル代入名: 特になし。

- 読み取る主なインスタンス属性: `self._update`
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、try 0

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。

上から順の処理

1. `self._update(...)` を実行する。

代表コード断片

```
def _on_mpc_state(self, msg: String):
    self._update('mpc_state', str(msg.data))
```

7.32 L364 関数 `DashboardNode._on_health`

- 行範囲: L364–L365
- 所属: `DashboardNode`
- 定義: `_on_health(self, msg: Float32)`
- `docstring`: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `_update`, `float`
- 戻り値の要点: 戻り値が無いか、`return` 文が明示されていない。
- 主なローカル代入名: 特になし。
- 読み取る主なインスタンス属性: `self._update`
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、try 0

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。

上から順の処理

1. `self._update(...)` を実行する。

代表コード断片

```
def _on_health(self, msg: Float32):
    self._update('system_health', float(msg.data))
```

7.33 L367 関数 DashboardNode._on_gps

- 行範囲: L367–L371
- 所属: DashboardNode
- 定義: `_on_gps(self, msg: NavSatFix)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `float`, `time`
- 戻り値の要点: 戻り値が無いか、`return` 文が明示されていない。
- 主なローカル代入名: 特になし。
- 読み取る主なインスタンス属性: `self._lock`, `self._state`
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、`try` 0

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。
- `with` 文は `context manager` に開始・終了処理を任せ、ファイルなどの資源を確実に解放する。

上から順の処理

1. `with` 文で `self._lock` を管理しながら処理する。
2. `self._state['gps_lat']` に `float(msg.latitude)` の結果を代入する。
3. `self._state['gps_lon']` に `float(msg.longitude)` の結果を代入する。
4. `self._state['ts']` に `time.time()` の結果を代入する。

代表コード断片

```
def _on_gps(self, msg: NavSatFix):
    with self._lock:
        self._state['gps_lat'] = float(msg.latitude)
        self._state['gps_lon'] = float(msg.longitude)
        self._state['ts'] = time.time()
```

7.34 L373 関数 DashboardNode._on_path

- 行範囲: L373–L377
- 所属: DashboardNode
- 定義: `_on_path(self, msg: Path)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `len`, `time`
- 戻り値の要点: 戻り値が無いか、`return` 文が明示されていない。

- 主なローカル代入名: 特になし。
- 読み取る主なインスタンス属性: `self._lock`, `self._state`
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、`try` 0

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。
- `with` 文は `context manager` に開始・終了処理を任せ、ファイルなどの資源を確実に解放する。

上から順の処理

1. `with` 文で `self._lock` を管理しながら処理する。
2. `self._state['path_points']` に `len(msg.poses)` の結果を代入する。
3. `self._state['ts']` に `time.time()` の結果を代入する。

代表コード断片

```
def _on_path(self, msg: Path):
    # Keep last path length as a simple sanity signal
    with self._lock:
        self._state['path_points'] = len(msg.poses)
        self._state['ts'] = time.time()
```

7.35 L379 関数 `DashboardNode._init_dummy`

- 行範囲: L379–L393
- 所属: `DashboardNode`
- 定義: `_init_dummy(self, path:str, rate_hz:float)`
- `docstring`: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `create_timer`, `error`, `get_logger`, `info`, `len`, `max`, `read_csv`, `to_dict`
- 戻り値の要点: 戻り値が無いか、`return` 文が明示されていない。
- 主なローカル代入名: `df`, `period`
- 読み取る主なインスタンス属性: `self._dummy_rows`, `self._tick_dummy`, `self.create_timer`, `self.get_logger`
- 更新する主なインスタンス属性: `self._dummy_idx`, `self._dummy_rows`, `self._dummy_timer`
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 1、ループ 0、`try` 1

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。
- `try` 文は例外が起き得る処理と、異常時または終了時の経路を分ける。

上から順の処理

1. 例外処理を伴う try ブロックを実行する。
2. Import 文を実行する。
3. df に pd.read_csv(path) の結果を代入する。
4. Exception を捕捉した場合:
5. self.get_logger().error(...) を実行する。
6. を返す。
7. 条件 df.empty を判定し、真なら内部処理を行う。
8. self.get_logger().error(...) を実行する。
9. を返す。
10. self._dummy_rows に df.to_dict(orient='records') の結果を代入する。
11. self._dummy_idx に 0 の結果を代入する。
12. period に 1.0 / max(rate_hz, 0.5) の結果を代入する。
13. self._dummy_timer に self.create_timer(period, self._tick_dummy) の結果を代入する。
14. self.get_logger().info(...) を実行する。

代表コード断片

```
def _init_dummy(self, path: str, rate_hz: float):
    try:
        import pandas as pd
        df = pd.read_csv(path)
    except Exception as exc:
        self.get_logger().error(f'Failed to load dummy_csv: {exc}')
        return
    if df.empty:
        self.get_logger().error('dummy_csv is empty.')
        return
    self._dummy_rows = df.to_dict(orient='records')
    self._dummy_idx = 0
    period = 1.0 / max(rate_hz, 0.5)
    self._dummy_timer = self.create_timer(period, self._tick_dummy)
    self.get_logger().info(f'Dummy mode enabled: {path} ({len(self._dummy_rows)} rows)')
```

7.36 L395 関数 DashboardNode._tick_dummy

- 行範囲: L395–L406
- 所属: DashboardNode
- 定義: _tick_dummy(self)
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: float, hasattr, items, len, time
- 戻り値の要点: 戻り値が無い、return 文が明示されていない。
- 主なローカル代入名: key, row, val

- 読み取る主なインスタンス属性: `self._dummy_idx`, `self._dummy_rows`, `self._lock`, `self._state`
- 更新する主なインスタンス属性: `self._dummy_idx`
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 1、ループ 1、`try` 1

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。
- `with` 文は `context manager` に開始・終了処理を任せ、ファイルなどの資源を確実に解放する。
- `try` 文は例外が起き得る処理と、異常時または終了時の経路を分ける。

上から順の処理

1. 条件 `not hasattr(self, '_dummy_rows')` or `not self._dummy_rows` を判定し、真なら内部処理を行う。
2. を返す。
3. `row` に `self._dummy_rows[self._dummy_idx]` の結果を代入する。
4. `self._dummy_idx` に `(self._dummy_idx + 1) % len(self._dummy_rows)` の結果を代入する。
5. `with` 文で `self._lock` を管理しながら処理する。
6. `row.items()` を順に走査し、各要素を `(key, val)` に入れて処理する。
7. 例外処理を伴う `try` ブロックを実行する。
8. `self._state[key]` に `float(val)` の結果を代入する。
9. `Exception` を捕捉した場合:
10. `self._state[key]` に `val` の結果を代入する。
11. `self._state['ts']` に `time.time()` の結果を代入する。

代表コード断片

```
def _tick_dummy(self):
    if not hasattr(self, '_dummy_rows') or not self._dummy_rows:
        return
    row = self._dummy_rows[self._dummy_idx]
    self._dummy_idx = (self._dummy_idx + 1) % len(self._dummy_rows)
    with self._lock:
        for key, val in row.items():
            try:
                self._state[key] = float(val)
            except Exception:
                self._state[key] = val
        self._state['ts'] = time.time()
```

7.37 L409 関数 main

- 行範囲: L409–L414
- 所属: module 直下
- 定義: `main()`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `DashboardNode`, `destroy_node`, `init`, `shutdown`, `spin`
- 戻り値の要点: 戻り値が無いか、`return` 文が明示されていない。
- 主なローカル代入名: `node`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、`try` 0

この定義を読むための Python 構文

- 特別な構文上の補足は少ない。

上から順の処理

1. `rclpy.init(...)` を実行する。
2. `node` に `DashboardNode()` の結果を代入する。
3. `rclpy.spin(...)` を実行する。
4. `node.destroy_node(...)` を実行する。
5. `rclpy.shutdown(...)` を実行する。

代表コード断片

```
def main():  
    rclpy.init()  
    node = DashboardNode()  
    rclpy.spin(node)  
    node.destroy_node()  
    rclpy.shutdown()
```

7.38 設定値と入力パラメータ

行	名前	既定値・補足
58	<code>host</code>	0.0.0.0
59	<code>port</code>	8080
60	<code>static_dir</code>	
61	<code>dummy_csv</code>	
62	<code>dummy_rate_hz</code>	5.0

7.39 ROS topic I/O

行	種別	topic	callback / 補足
117	subscription	/planner/speed_cmd	self._on_speed_cmd
118	subscription	/planner/upper_speed_cmd	self._on_upper_speed_cmd
119	subscription	/vehicle/speed_kmh	self._on_speed_meas
120	subscription	/planner/throttle_cmd_pct	self._on_throttle_cmd
121	subscription	/planner/drive_mode	self._on_drive_mode
122	subscription	/vehicle/batt_soc	self._on_soc
123	subscription	/vehicle/batt_temp_c	self._on_tb
124	subscription	/vehicle/batt_current_a	self._on_ibatt
125	subscription	/vehicle/batt_voltage_v	self._on_vbatt
126	subscription	/vehicle/s_kmh	self._on_s_kmh
127	subscription	/planner/status	self._on_status
128	subscription	/planner/env	self._on_env
129	subscription	/planner/metrics	self._on_metrics
130	subscription	/planner/upper_plan	self._on_upper_plan
131	subscription	/planner/lower_plan	self._on_lower_plan
132	subscription	/system/state	self._on_sys_state
133	subscription	/system/diag	self._on_sys_diag
134	subscription	/system/mpc_state	self._on_mpc_state
135	subscription	/system/health	self._on_health
136	subscription	/sim/gps	self._on_gps
137	subscription	/planner/trajectory	self._on_path

8 処理の流れ

1. 初期化時に設定値や入力パスを読み込む。
2. publisher / subscription / timer を準備する。
3. timer callback 周期で主処理を進める。

9 このファイルの位置づけ

この資料群では、`mpc_solarcar/dashboard_node.py` を **runtime node** の中核として扱う。したがって、ここで扱う処理は単独で完結せず、前段の `profile / launch / telemetry` と後段の `planner / logger / report` へ繋がる。